DAY 23 | HANDS-ON ACTIVITIES

Industry Use Cases and Agentic Security: Problem Statements

Seven problems. Every notebook cell runs offline; pip-audit and npm audit need internet access. All problems are NEW (v4).

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Use Case Mapper NEW (v4)

Level: Warm-up | **Concepts:** GenAI by industry

Given ten short business requests (for example 'summarise outage chats', 'extract fields from claim forms'), label each with its industry, the GenAI pattern (summarise, extract, classify, generate, search) and the main risk.

Sample run / expected result

```
outage chats  -> telecom | summarise | wrong info in an incident
claim fields  -> BFSI    | extract   | personal data, accuracy
```

Hints

- The risk usually comes from the data or from what happens when the answer is wrong.

Problem 2: Maturity Self-Assessment NEW (v4)

Level: Easy | **Concepts:** maturity levels

Use the notebook's scorer to place SmartDesk on the maturity ladder. Answer the ten yes or no questions honestly, then list the three gaps that block the next level.

Sample run / expected result

```
Score: 4/10 -> Pilot
Gaps: no golden set, no cost budget, no on-call owner
```

Hints

- Be strict: 'we could add it' is a no.

Problem 3: Pilot to Production Plan NEW (v4)

Level: Medium | **Concepts:** what moves a pilot to production

Write a one-page plan to move SmartDesk's summariser from pilot to production: owner, success metric with a baseline, eval plan, security review, cost model and runbook. Keep each item to two lines.

Sample run / expected result

```
Metric: average handling time, baseline 9 min, target 7 min
Evals: 50-ticket golden set, gate at 90%
```

Hints

- Use numbers you can measure; mark any estimate as an estimate.

Problem 4: Scoped Token Checker NEW (v4)

Level: Medium | **Concepts:** agent identity, scopes, expiry

Write check(token, needed_scope) that returns True only if the token is valid, not expired, has the right audience and contains the scope. Test it with a valid token, an expired token, a token for another audience and a token missing the scope.

Sample run / expected result

```
valid + tickets:read   -> True
expired                -> False (expired)
missing tickets:close  -> False (scope)
```

Hints

- Deny by default: any exception means False.
- Keep token lifetime to minutes, not days.

Problem 5: Tool Permission Gate NEW (v4)

Level: Medium | **Concepts:** deny by default, human approval

Build a gate in front of an agent's tools. Unknown tools are denied, read tools are allowed, money-moving tools need approval. Feed it a ticket containing a hidden instruction to delete a customer and show it is blocked and logged.

Sample run / expected result

```
search_kb        -> ALLOW
refund_customer  -> NEEDS_APPROVAL
delete_customer  -> DENY (logged)
```

Hints

- Log every decision with the tool name, arguments and reason.

Problem 6: Scan and Report the Starter Repo NEW (v4)

Level: Core (assignment) | **Concepts:** bandit, pip-audit, npm audit

Clone the starter repo (shared/PatchAndModernise_Starter). Run the tests, then bandit, pip-audit and npm audit. Log every finding in the vulnerability report template with severity, file and a proposed fix. Mark verified as 'no' until a test proves the fix.

Sample run / expected result

```
pytest: 14 passed
bandit: 4 issues
pip-audit: at least 1 vulnerable package
npm audit: several advisories
```

Hints

- Save raw scanner output as evidence.
- Ask: is this reachable from user input?

Problem 7: Characterise, then Modernise NEW (v4)

Level: Stretch | **Concepts:** characterisation tests, small steps

Run the characterisation tests for legacy_csv_import.py. Use Codex to migrate it one step at a time (csv module, timezone-aware dates, no global state). Run the tests after every step and commit each green step.

Sample run / expected result

```
step 1 csv module          : 7 passed
step 2 timezone-aware date : 7 passed
step 3 no global state     : 7 passed
```

Hints

- If a test fails, undo the step instead of changing the test.
- Keep each Codex request small.